



Universidad
Rafael Landívar

Tradición Jesuita en Guatemala

MANUAL TÉCNICO

Generador de Imágenes por Capas

Universidad Rafael Landívar – Campus Quetzaltenango
Estructura de Datos | Proyecto Final

Catedrático: Ing. Juan Pablo Valiente

Proyecto Final | Mayo 2026

Moshé Arenz Peláez Virula

ÍNDICE

INTRODUCCIÓN	1
1. ENTORNO DE DESARROLLO	1
2. ESTRUCTURA DEL PROYECTO.....	1
3. DEFINICIÓN DE STRUCTS (Structs.h)	2
4. ESTRUCTURAS DE DATOS	3
4.1. Matriz Dispersa (MatrizDispersa)	3
4.2. Árbol Binario de Búsqueda de Capas (ABBCapas)	3
4.3. Lista Circular Doblemente Enlazada (ListaCircular).....	4
4.4. Árbol Binario de Búsqueda de Usuarios (ABBUusuarios)	4
5. CLASES UTILITARIAS (UTILS).....	5
5.1. CargaMasiva	5
5.2. Graphviz.....	5
5.3. GeneradorImágenes.....	6
6. FLUJO GENERAL DEL SISTEMA	7
7. COMPILACIÓN Y EJECUCIÓN	8
7.1. CMakeLists.txt	8
7.2. Working Directory	8
7.3. Dependencias externas.....	8
8. CONTROL DE VERSIONES	9
Commits principales realizados:	9

INTRODUCCIÓN

El presente manual técnico documenta la arquitectura, organización y funcionamiento interno del sistema Generador de Imágenes por Capas, desarrollado en C++ con el propósito de aplicar de forma práctica los conceptos fundamentales de las estructuras de datos. La aplicación recibe archivos de entrada con extensiones **.cap**, **.im** y **.usr**, los procesa automáticamente al iniciar, y permite generar imágenes, gestionar usuarios e imágenes, y visualizar el estado interno de la memoria mediante reportes gráficos.

Todas las estructuras fueron implementadas desde cero sin el uso de librerías externas. Se utilizó una Matriz Dispersa para representar los píxeles de cada capa almacenando únicamente los nodos con color, un Árbol Binario de Búsqueda para organizar las capas por ID y los usuarios por nombre, y una Lista Circular Doblemente Enlazada para gestionar las imágenes del sistema. Para los reportes gráficos se utilizó Graphviz generando imágenes .png, y para las imágenes de píxeles el formato PPM, compatible con macOS, Linux y Windows.

Este manual está dirigido a desarrolladores que necesiten comprender, mantener o extender el sistema, asumiendo conocimiento previo de C++, estructuras de datos y CMake.

1. ENTORNO DE DESARROLLO

Elemento	Detalle
Lenguaje	C++
IDE	CLion
Sistema de build	CMake 4.2
Sistema Operativo	macOS (ARM64)
Herramienta de reportes	Graphviz 12.2.1
Control de versiones	Git + GitHub
Repositorio	PROYECTO_FINAL_EDD https://github.com/Arenzzzz/PROYECTO_FINAL_EDD.git

2. ESTRUCTURA DEL PROYECTO

El proyecto está organizado en carpetas según su responsabilidad.

```

PROYECTO_FINAL/
├── CMakeLists.txt ← Configuración de compilación
├── main.cpp ← Punto de entrada y menús
├── Structs.h ← Definición de todos los nodos
├── ESTRUCTURAS/ ← Implementación de estructuras de datos
│   ├── MatrizDispersa.h / .cpp
│   ├── ABBCapas.h / .cpp
│   ├── ListaCircular.h / .cpp
│   └── ABBUuarios.h / .cpp
├── UTILS/ ← Utilidades del sistema
│   ├── CargaMasiva.h / .cpp
│   ├── Graphviz.h / .cpp
│   └── GeneradorImágenes.h / .cpp
├── ARCHIVOS/ ← Archivos de entrada (.cap .im .usr)
└── REPORTES/ ← Imágenes generadas (.png .ppm)
  
```

3. DEFINICIÓN DE STRUCTS (Structs.h)

Archivo central que define todos los nodos utilizados por las estructuras de datos. Al estar en la raíz del proyecto, todos los archivos lo pueden incluir sin conflictos.

Struct	Campos principales	Descripción
NodoMatriz	fila, columna, color, *derecha, *abajo	Nodo de la matriz dispersa
NodoCapa	*refCapa, *siguiente	Referencia a capa en el ABB
NodoABBCapa	id, *matriz, *matrizObj, *izq, *der	Nodo del ABB de capas
Imagen	id, listaCapas	Imagen con su lista de capas
NodoImagen	Imagen, *siguiente, *anterior	Nodo de la lista circular
NodoImgUsuario	idImagen, *siguiente	Imagen en lista de usuario
NodoABBUusuario	nombre, *listImágenes, *izq, *der	Nodo del ABB de usuarios

NOTA DEL DISEÑO: la lista de capas de cada imagen almacena un puntero (NodoABBCapa*) al nodo del ABB en lugar de una copia. Esto ahorra memoria y garantiza que cualquier modificación en el ABB se refleje automáticamente en todas las imágenes que referencian esa capa.

4. ESTRUCTURAS DE DATOS

4.1. Matriz Dispersa (MatrizDispersa)

Almacena los píxeles de una capa. Solo guarda nodos con color, optimizando memoria para capas con dimensiones variables. La cabeza apunta al primer nodo de fila. Cada fila tiene una lista de columnas enlazadas horizontalmente.

Método	Tipo	Descripción
MatrizDispersa()	Constructor	Inicializa cabeza en nullptr
InsertarPixel(fila, col, color)	void	Insertar píxel en posición indicada
obtenerColor(fila, col)	string	Retorna cabeza de la matriz
getCabeza()	NodoMatriz*	Retorna cabeza de la matriz
buscarOCrearFila(fila)	NodoMatriz*	Privado – busca o crea nodo fila
buscarOCrearColumna(...)	NodoMatriz*	Privado – busca o crea nodo columna

4.2. Árbol Binario de Búsqueda de Capas (ABBCapas)

Organiza todas las capas del sistema por ID. IDs menores van al subárbol izquierdo, mayores al derecho. IDs duplicados se ignoran. Cada nodo contiene un puntero a su MatrizDispersa.

Método	Tipo	Descripción
ABBCapas()	Constructor	Inicializa raíz en nullptr
insertarCapa(id)	void	Inserta nueva capa en el árbol
buscarCapa(id)	NodoABBCapa*	Retorna nodo de la capa por ID
insertarPixelEnCapa(id, f, c, color)	void	Inserta píxel en la capa indicada
getRaiz()	NodoABBCapa*	Retorna raíz del árbol
insertar(nodo, id)	NodoABBCapa*	Privado – inserción recursiva
buscar(nodo, id)	NodoABBCapa*	Privado – búsqueda recursiva

4.3. Lista Circular Doblemente Enlazada (ListaCircular)

Almacena las imágenes del sistema ordenadas por ID. El último nodo apunta al primero y el primero al último. Cada imagen contiene su propia lista de capas con punteros al ABB.

Método	Tipo	Descripción
ListaCircular()	Constructor	Inicializa cabeza en nullptr
insertarImagen(id)	void	Inserta imagen ordenada, valida duplicados
buscarImagen(id)	NodoImagen*	Retorna nodo de imagen por ID
eliminarImagen(id)	void	Elimina imagen manteniendo estructura circular
agregarCapaAlmagen(idImg, *capa)	void	Agrega referencia de capa a imagen
getCabeza()	NodoImagen*	Retorna cabeza de la lista
estaVacia()	bool	Retorna true si la lista está vacía

4.4. Árbol Binario de Búsqueda de Usuarios (ABBUusuarios)

Organiza los usuarios por nombre en orden alfabético. Cada usuario tiene una lista enlazada simple con los IDs de sus imágenes.

Método	Tipo	Descripción
ABBUusuarios()	Constructor	Inicializa raíz en nullptr
insertarUsuario(nombre)	void	Inserta usuario en el árbol
buscarUsuario(nombre)	NodoABBUusuario*	Retorna nodo del usuario
eliminarUsuario(nombre)	void	Elimina usuario del árbol
agregarImagenAUusuario(nombre, id)	void	Agrega imagen a lista del usuario
getRaiz()	NodoABBUusuario*	Retorna raíz del árbol
minimoNodo(nodo)	NodoABBUusuario*	Privado – nodo con valor mínimo

5. CLASES UTILITARIAS (UTILS)

5.1. CargaMasiva

Lee los archivos de entrada y puebla estructuras de datos en memoria. El orden de carga es obligatorio: **capas** → **imágenes** → **usuarios**.

Método	Descripción
cargarCapas(ruta, arbolCapas)	Lee .cap e inserta capas y píxeles en el ABB
cargarImagenes(ruta, lista, árbol)	Lee .im e inserta imágenes en la lista circular
cargarUsuarios(ruta, árbol, lista)	Lee .usr e inserta usuarios en el ABB
trim(str)	Privado – elimina espacios y tabs al inicio y al final

Lógica de parseo del archivo .cap

- **Línea con '{'**: extrae el ID antes de la llave y crea la capa.
- **Línea con '}'**: cierra la capa actual.
- **Línea con ','**: parsea fila, columna y color de píxel.
- **Si no hay ID antes del '{'**: asigna el siguiente ID disponible.

5.2. Graphviz

Genera archivos **.dot** y los procesa con el comando **'dot'** para producir imágenes **.png** en la carpeta REPORTES/.

Método	Archivo generado
graficarArbolCapas(raiz)	REPORTES/árbol_capas/png
graficarListaImagenes(cabeza)	REPORTES/lista_imagenes.png
graficarCapa(nodo)	REPORTES/capa_[id].png
graficarImagenYArbol(imgNodo, raiz)	REPORTES/imagen_arbol_[id].png
graficarArbolUsuarios(raiz)	REPORTES/arbol_usuarios.png
ejecutarDot(dot, salida)	Privado – ejecuta comando dot del sistema

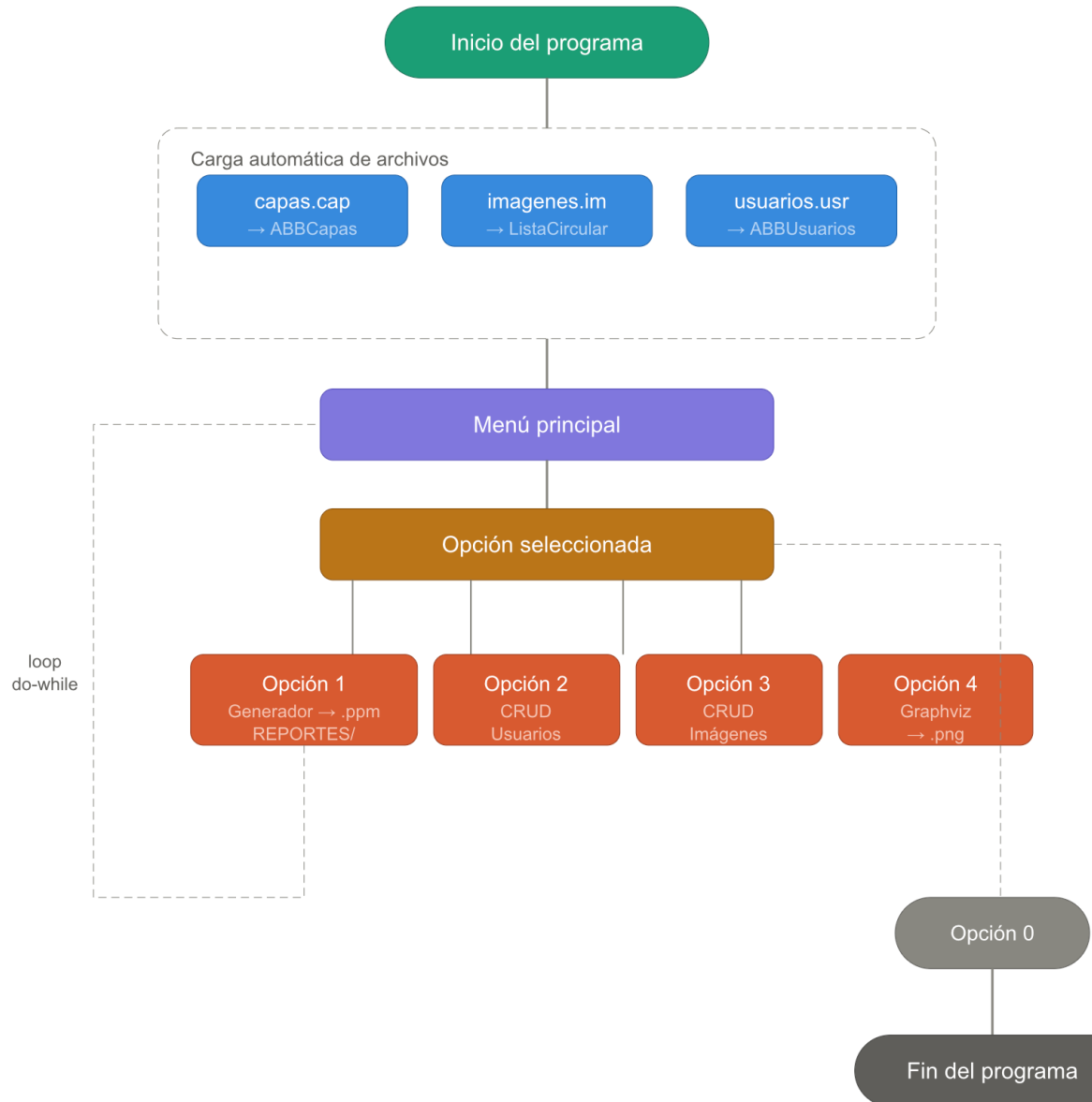
5.3. GeneradorImágenes

Genera imágenes en formato PPM superponiendo capas en un canvas. El canvas es un mapa de (fila, columna) → color. Si dos capas tienen un píxel en la misma posición, la última capa agregada sobrescribe.

Método	Archivo generado
generarPorRecorridoLimitado(raiz, N, tipo)	REPORTES/imagen_recorrido.ppm
generarPorListaImágenes(imgNodo)	REPORTES/imagen_[id].ppm
generarPorCapa(nodo)	REPORTES/imagen_capa_[id].ppm
generarPorUsuario(usuario, id, lista)	REPORTES/imagen_[id].ppm

6. FLUJO GENERAL DEL SISTEMA

El siguiente diagrama describe el flujo de ejecución del sistema:



7. COMPILACIÓN Y EJECUCIÓN

7.1. CMakeLists.txt

El archivo CMakeLists.txt en la raíz configura la compilación. Incluye todos los archivos fuente y crea automáticamente la carpeta REPORTES/:

```
cmake_minimum_required(VERSION 4.2)
project(PROYECTO_FINAL)

set(CMAKE_CXX_STANDARD 20)

add_executable(PROYECTO_FINAL main.cpp
    ESTRUCTURAS/MatrizDispersa.cpp
    ESTRUCTURAS/MatrizDispersa.h
    ESTRUCTURAS/ABBCapas.cpp
    ESTRUCTURAS/ABBCapas.h
    ESTRUCTURAS/ListaCircular.cpp
    ESTRUCTURAS/ListaCircular.h
    ESTRUCTURAS/ABBUuarios.cpp
    ESTRUCTURAS/ABBUuarios.h
    UTILS/CargaMasiva.cpp
    UTILS/CargaMasiva.h
    UTILS/Graphviz.cpp
    UTILS/Graphviz.h
    Structs.h
    UTILS/GeneradorImagenes.cpp
    UTILS/GeneradorImagenes.h
)

file(MAKE_DIRECTORY ${CMAKE_SOURCE_DIR}/REPORTES)
```

7.2. Working Directory

El Working Directory debe configurarse en la raíz del proyecto para que las rutas relativas de los archivos ARCHIVOS/ y REPORTES/ funcionen correctamente. En CLion: **Run** → **Edit Configurations** → **Working Directory**.

7.3. Dependencias externas

Dependencia	Versión	Uso
Graphviz	12.2.1 +	Generación de reportes .png
CMake	4.2 +	Sistema de build
C++ STL	C++ 20	Map, vector, fstream, sstream, functional

8. CONTROL DE VERSIONES

El Proyecto utiliza Git con GitHub para el control de versiones.

Rama	Descripción
main	Código estable y funcional. Versión final del proyecto
Dev	Rama de desarrollo donde se implementaron los cambios

Commits principales realizados:

Commit	Descripción
1	Estructura base del proyecto
2	Matriz Dispersa implementada y prueba de esta
3	ABBCapas implementado y probado
4	ListaCircular implementado y probada
5	ABBUuarios implementado y probado
6	CargaMasiva implementada y probada
7	Menú principal con carga automática
8	Graphviz implementado y probado
9	GeneradorImágenes implementado y probado
10	Uso del sistema con archivos de prueba
11	Merge dev → main Versión final